TECH CHALLENGE · FASE 4

Plataforma de Feedback de Aulas

Cloud Computing, Serverless e Deploy de Aplicações na AWS
Documentação da solução

Sumário

1. Visão geral 2

2. Atendimento aos requisitos 2

3. Arquitetura da solução 3

4. Modelo de cloud escolhido 6

5. Repositórios e funções 7

6. Segurança e governança de acesso 10

7. Monitoramento 11

8. Deploy 11

Nuvem: **AWS**, região **sa-east-1 (São Paulo)** · Infraestrutura como código: **Terraform** · Funções: **AWS Lambda (Java 21)** · CI/CD: **GitHub Actions**

1. Visão geral

Problema. Para garantir a qualidade dos cursos on-line, os estudantes precisam avaliar as aulas e os administradores precisam acompanhar rapidamente a satisfação dos alunos: saber na hora quando chega uma avaliação crítica e receber periodicamente um resumo das avaliações.

Solução. Uma aplicação **100% serverless na AWS**, com três funções Lambda, cada uma com uma única responsabilidade:

Função	O que faz	Quando roda
Ingestão	Recebe a avaliação do aluno, valida, calcula a urgência e grava no banco	A cada `POST /evaluations`
Notificação	Envia e-mail aos administradores sobre a avaliação crítica	Quando entra uma avaliação de urgência alta (nota 0 a 3)
Relatório	Envia aos administradores o relatório da semana, com média e contagens	Toda segunda-feira às 08:00 (Brasília)

Toda a infraestrutura é criada por código (Terraform) e o deploy é automático pelo GitHub Actions, sem chaves de acesso fixas. O projeto está dividido em quatro repositórios: um de infraestrutura e um por função.

2. Atendimento aos requisitos

Requisito do enunciado	Como foi atendido	Seção
Ambiente de nuvem com segurança dos dados e governança de acesso	Cognito, autorizador JWT, IAM com permissões mínimas, OIDC, criptografia, PITR, LGPD nos logs	6
Componentes de suporte (bancos de dados etc.)	DynamoDB, SES, SQS (DLQ), EventBridge, Cognito	3.1
Deploy automatizado dos componentes atualizáveis	GitHub Actions: Terraform na infra, update-function-code nas funções	8
Aplicação monitorada	CloudWatch: logs e métricas das funções, API, DynamoDB e DLQ	7
Notificações automáticas aos administradores para problemas críticos	E-mail imediato para cada avaliação de urgência alta, com descrição, urgência e data de envio	3.2, 5.3
Relatório semanal com média de avaliações	Toda segunda às 08:00: média, quantidade por dia e por urgência, e a lista com descrição, urgência e data de envio	3.2, 5.4
Serverless e em cloud	AWS Lambda e serviços gerenciados da AWS	4
Mínimo de 2 funções, com responsabilidade única	3 funções, cada uma com repositório, pipeline e permissões próprios	5
Repositório aberto com o código-fonte	4 repositórios públicos no GitHub	5
Arquitetura, deploy, monitoramento e documentação das funções	Este documento e o README de cada repositório	3, 5, 7, 8

3. Arquitetura da solução

Todas as peças dentro da conta AWS e como se conectam. Os rótulos **F1**, **F2** e **F3** nas setas indicam o fluxo de cada passo (seção 3.2). Fora da AWS ficam os usuários, o GitHub Actions (deploy) e o Terraform Cloud (estado da infraestrutura).

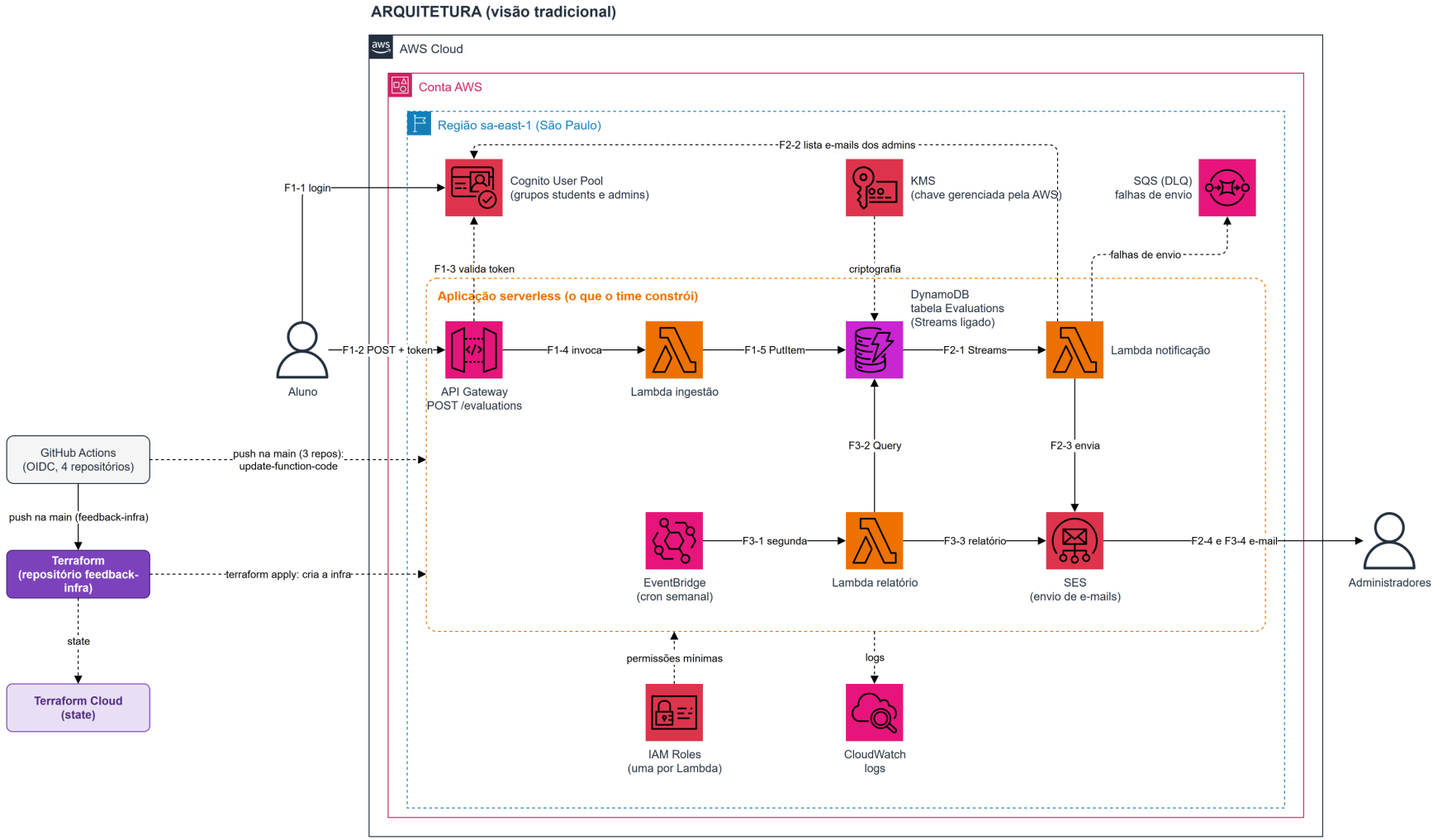


Figura 1 – Arquitetura da solução na AWS.

3.1 Componentes

Camada	Serviço AWS	Papel na solução
Acesso	**Cognito** (User Pool)	Login por e-mail; grupos **students** (alunos) e **admins**. Emite o token JWT usado na API. Só o administrador cria contas.
Acesso	**API Gateway** (HTTP API)	Rota `POST /evaluations` com autorizador JWT do Cognito; limite de 1 requisição/s.
Processamento	**Lambda** (3 funções)	`feedback-ingestion`, `feedback-notification`, `feedback-report` — Java 21, 512 MB.
Dados	**DynamoDB**	Tabela `Evaluations` sob demanda, índice `by-day` para o relatório, Streams para disparar a notificação, backup contínuo (PITR).
Integração	**SES**	Envia o e-mail de urgência e o relatório semanal.
Integração	**EventBridge**	Dispara o relatório toda segunda às 08:00 (Brasília).
Integração	**SQS** (DLQ)	`feedback-notification-dlq`: guarda o aviso que falhou após as tentativas, para não se perder.
Governança	**IAM**	Uma role por função com permissões mínimas; roles das pipelines via OIDC.
Governança	**KMS**	Chave gerenciada pela AWS (`aws/dynamodb`) cifra os dados em repouso.
Governança	**CloudWatch**	Logs de cada função (14 dias) e métricas automáticas.
Entrega	Terraform + Terraform Cloud	Define todos os recursos; o Terraform Cloud guarda o estado.
Entrega	GitHub Actions	Testa e publica o código das funções e aplica o Terraform.

3.2 Fluxos

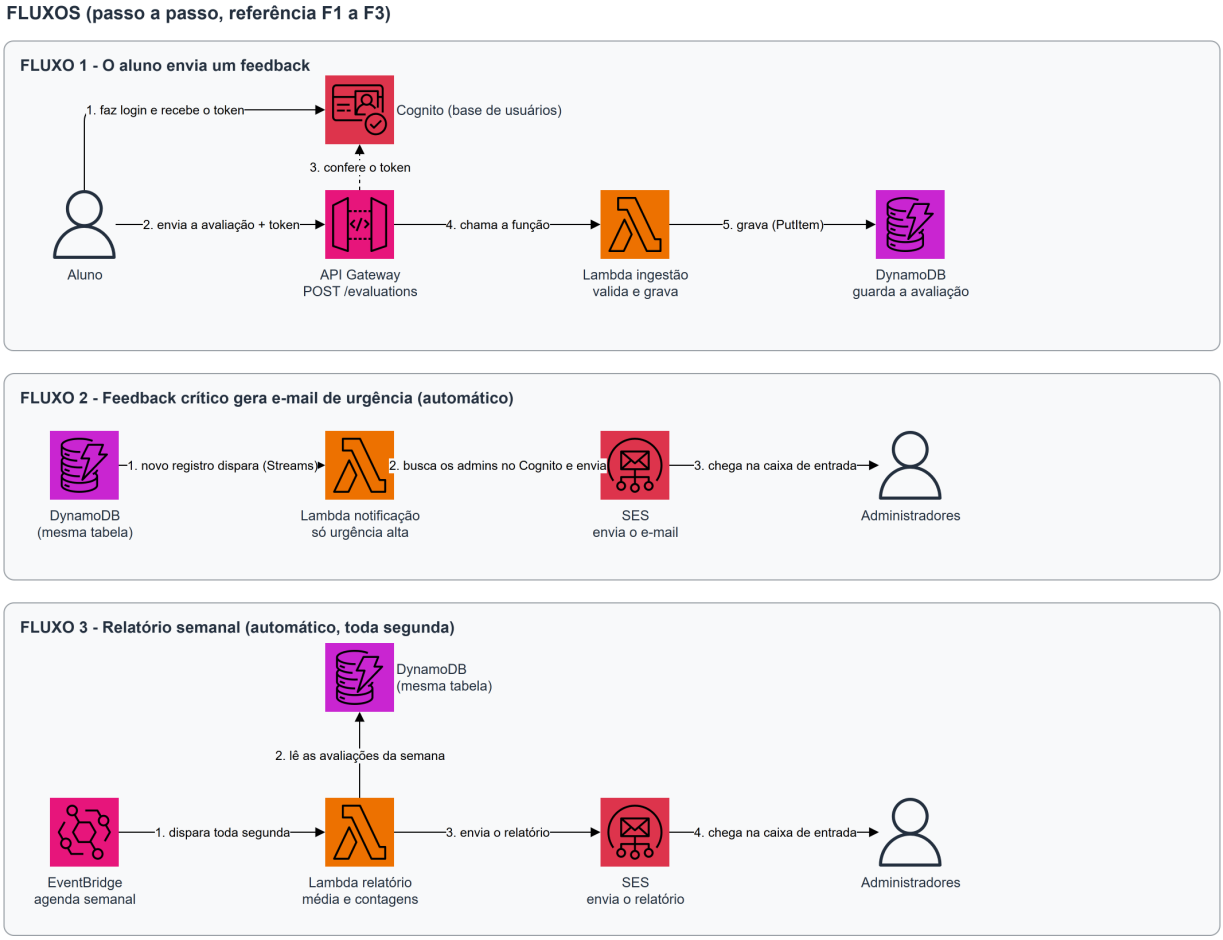


Figura 2 – Os três fluxos da aplicação, passo a passo.

Fluxo	Começa com	O que acontece	Resultado
F1 Envio do feedback	Aluno chama `POST /evaluations` com o token	O API Gateway confere o token no Cognito (sem token: 401). A ingestão valida, calcula a urgência e grava.	Resposta `201` com o `evaluationId`
F2 Aviso de urgência	Registro novo no DynamoDB Streams	Um filtro só invoca a função para inserções com urgência **high**. A notificação busca os admins no Cognito e envia pelo SES. Se falhar: 2 novas tentativas e depois a DLQ.	E-mail para cada administrador
F3 Relatório semanal	EventBridge, segunda 08:00 (`cron(0 11 ? * MON *)` UTC)	O relatório consulta os 7 dias anteriores no índice `by-day` (uma consulta por dia, sem varrer a tabela) e calcula média e contagens.	Relatório para cada administrador

3.3 Decisões de arquitetura

Decisão	Motivo
HTTP API em vez de REST API	Mais barata e suficiente para uma rota com autorizador JWT.
Notificação disparada pelo DynamoDB Streams	Desacopla ingestão e notificação: o aluno não espera o e-mail, e uma falha no e-mail não impede a gravação.
Filtro no gatilho do Streams	A função só roda para avaliações críticas: menos execuções, menos custo.
Índice `by-day`	O relatório lê só os dias da semana, sem `Scan`; o índice não guarda o `userId`.
Fila de falhas (DLQ)	Nenhum aviso crítico se perde.
Uma função e um repositório por responsabilidade	Cada função tem código, testes, pipeline e permissões próprios e é atualizada sem afetar as outras.
Arquitetura hexagonal nas funções	As regras de negócio não dependem da AWS: fáceis de testar.

4. Modelo de cloud escolhido

Nuvem pública (AWS) com modelo **serverless**: o código roda como funções (**FaaS**, AWS Lambda) e o restante são **serviços gerenciados** (API Gateway, Cognito, DynamoDB, SES, SQS, EventBridge, CloudWatch). A equipe não cria nem administra servidores.

Característica	Por que foi escolhida
Paga só pelo uso	Nenhum servidor ligado parado. Com créditos limitados, o custo acompanha o uso real.
Sem servidor para administrar	A AWS cuida de máquinas, sistema operacional, atualizações, escala e disponibilidade.
Escala automática	Cada função escala sozinha conforme as requisições ou eventos.
Orientado a eventos	Cada função é disparada por um evento: requisição HTTP, registro novo no banco ou agendamento.
Região sa-east-1	Dados guardados no Brasil e menor latência para os usuários.

Responsabilidade compartilhada: a AWS responde pela infraestrutura física, servidores, rede e sistema operacional; a equipe responde pelo código, permissões, configuração dos serviços e proteção dos dados (seção 6).

Controle de custos: tudo cobrado por uso; limite de 1 req/s na API; capacidade máxima no DynamoDB (o excesso é recusado, não cobrado); poucas novas tentativas no gatilho da notificação; logs por só 14 dias; alerta mensal de orçamento (AWS Budgets) configurado no console.

5. Repositórios e funções

Repositório	Conteúdo	Endereço
feedback-infra	Infraestrutura (Terraform)	github.com/biadevcosta/feedback-infra
feedback-lambda-ingestion	Função de ingestão	github.com/biadevcosta/feedback-lambda-ingestion
feedback-lambda-notification	Função de notificação	github.com/biadevcosta/feedback-lambda-notification
feedback-lambda-report	Função de relatório	github.com/biadevcosta/feedback-lambda-report

A infraestrutura cria as funções com um código provisório; cada repositório de função publica o seu código real. Assim, infraestrutura e código evoluem de forma independente.

5.1 feedback-infra — infraestrutura

github.com/biadevcosta/feedback-infra · Terraform, estado no Terraform Cloud (organização `feedback-tc-ana`). Todos os recursos usam o prefixo `feedback-`.

Arquivo	O que cria
`api.tf`	HTTP API, rota `POST /evaluations`, autorizador JWT do Cognito, limite de 1 req/s
`cognito.tf`	User Pool (login por e-mail, sem cadastro aberto), cliente do app, grupos students e admins
`dynamodb.tf`	Tabela `Evaluations` (sob demanda, capacidade máxima, PITR, criptografia, Streams) e índice `by-day`
`lambdas.tf`	As 3 funções, a permissão de cada uma, o gatilho filtrado do Streams e a DLQ
`scheduler.tf`	Regra do EventBridge que dispara o relatório
`ses.tf`	Identidades do SES: remetente e administradores
`iam.tf`	Provedor OIDC do GitHub e roles das pipelines
`modules/lambda-function/`	Módulo reutilizável: função Java 21 + role própria + grupo de logs (14 dias)
`.github/workflows/`	`ci.yml` (pull request: fmt, validate, plan) e `deploy.yml` (main: apply)
`docs/`	Diagrama de arquitetura (PNG e arquivo editável do draw.io)

Contrato de dados

As três funções compartilham a tabela `Evaluations` (chave `evaluationId`):

Atributo	Tipo	Descrição
`evaluationId`	string	UUID v4
`userId`	string	`sub` do usuário no Cognito
`description`	string	Texto da avaliação (até 1000 caracteres)
`score`	number	Inteiro de 0 a 10
`urgency`	string	`high` (0–3), `medium` (4–6), `low` (7–10)
`submittedAt`	string	Data e hora de envio (ISO 8601, UTC)
`day`	string	Dia do envio no horário de Brasília (AAAA-MM-DD) — chave do índice `by-day`

Padrão das três funções

- **Java 21** com Maven; o build gera um único jar para a Lambda.
- **Arquitetura hexagonal**: `domain/` (regras, sem AWS) · `application/` (caso de uso e portas) · `adapter/in` (evento da AWS → comando) · `adapter/out` (DynamoDB, Cognito, SES) · `config/` (variáveis de ambiente). Testes **ArchUnit** garantem essa separação.
- **Testes unitários** só com mocks; o build falha abaixo de **80% de cobertura** (JaCoCo).
- **Pipeline** `ci-cd.yml`: pull request → build e testes; push na main → testes e publicação com `aws lambda update-function-code`.
- **LGPD**: logs só com ids, contagens e tipos de erro.

5.2 feedback-lambda-ingestion — ingestão

github.com/biadevcosta/feedback-lambda-ingestion · Recebe a avaliação do aluno e grava no banco.

Gatilho	API Gateway (HTTP API), depois do autorizador JWT
Configuração	512 MB · timeout 20 s · variável `TABLE_NAME`
Código	Domínio: Evaluation, Score, Description, Urgency · Caso de uso: SubmitEvaluationUseCase · Entrada: API Gateway · Saída: DynamoDB

Requisição — o `userId` vem do token, nunca do corpo:

```
POST /evaluations
Authorization: Bearer <token JWT do Cognito>

{ "description": "texto não vazio, até 1000 caracteres", "score": 7 }
```

Status	Corpo	Quando
201	`{"evaluationId": "<uuid>"}`	Avaliação gravada
400	`{"message": "<erro>"}`	JSON inválido, campo faltando, nota fora de 0–10, descrição vazia ou longa
401	`{"message": "Unauthorized"}`	Sem token válido
500	`{"message": "Could not save the evaluation"}`	Falha ao gravar

Regra de urgência: nota 0–3 = `high` (gera e-mail imediato) · 4–6 = `medium` · 7–10 = `low` (ambas só entram no relatório).

> **Contrato do enunciado.** O enunciado sugere `POST /avaliação` com `descricao` e `nota`. A equipe padronizou o código em inglês: `POST /evaluations` com `description` (descrição) e `score` (nota, inteiro de 0 a 10) — mesmo significado e mesmas regras.

5.3 feedback-lambda-notification — notificação

github.com/biadevcosta/feedback-lambda-notification · Avisa os administradores por e-mail quando chega uma avaliação crítica.

Gatilho	DynamoDB Streams filtrado (`INSERT` com `urgency = high`), 1 registro por vez, 2 novas tentativas, depois DLQ `feedback-notification-dlq`
Configuração	512 MB · timeout 30 s · variáveis `USER_POOL_ID`, `ADMIN_GROUP`, `SES_SENDER`
Código	Domínio: CriticalEvaluation, UrgencyEmailRenderer · Caso de uso: NotifyAdminsUseCase · Entrada: DynamoDB Streams · Saída: Cognito (lista admins), SES (e-mail)
E-mail	**Descrição**, **urgência** e **data de envio** (horário de Brasília). Um e-mail por administrador ativo e com e-mail verificado; o texto do aluno é escapado no HTML.
Falhas	Não são engolidas: o gatilho tenta de novo e, se continuar falhando, o evento fica na DLQ.

5.4 feedback-lambda-report — relatório

github.com/biadevcosta/feedback-lambda-report · Gera e envia o relatório semanal.

Gatilho	EventBridge, toda segunda às 08:00 de Brasília (`cron(0 11 ? * MON *)`, UTC)
Período	Os 7 dias completos anteriores (segunda a domingo)
Configuração	512 MB · timeout 30 s · variáveis `TABLE_NAME`, `INDEX_NAME`, `USER_POOL_ID`, `ADMIN_GROUP`, `SES_SENDER`, `DAYS`
Código	Domínio: ReportPeriod, WeeklyReportCalculator, ReportEmailRenderer · Caso de uso: SendWeeklyReportUseCase · Entrada: EventBridge · Saída: DynamoDB (índice by-day), Cognito, SES
Relatório	**Média** das notas (1 casa decimal) · **quantidade por dia** (os 7 dias, inclusive os sem avaliação) · **quantidade por urgência** · lista com **descrição, urgência e data de envio**
Observação	É enviado mesmo numa semana sem avaliações, para os administradores saberem que rodou. Pode ser gerado na hora com `aws lambda invoke --function-name feedback-report out.json`.

6. Segurança e governança de acesso

Autenticação e autorização

- Login pelo **Cognito**, sem cadastro aberto: só o administrador cria usuários. Senha com no mínimo 8 caracteres (maiúsculas, minúsculas e números); tokens valem 1 hora.
- A API só aceita requisições com token JWT válido do Cognito.
- Grupos **students** e **admins**; os e-mails e relatórios vão só para o grupo admins.

Permissões mínimas (IAM) — uma role por função

Função	Pode	Não pode
Ingestão	`dynamodb:PutItem` na tabela	Ler a tabela, enviar e-mail, acessar o Cognito
Notificação	Ler o Streams da tabela, `cognito-idp:ListUsersInGroup`, `ses:SendEmail`, `sqs:SendMessage` (DLQ)	Ler ou gravar a tabela diretamente
Relatório	`dynamodb:Query` só no índice by-day, `cognito-idp:ListUsersInGroup`, `ses:SendEmail`	Fazer Scan, acessar a tabela base, gravar dados

Cada função também só escreve no próprio grupo de logs.

Pipelines sem chaves fixas (OIDC)

Nenhuma chave da AWS fica no GitHub. As pipelines se autenticam por OIDC e assumem roles que só aceitam o repositório e a branch certos:

Role	Quem usa	Permissão
`feedback-github-infra-plan`	Pull requests do feedback-infra	Somente leitura
`feedback-github-infra-apply`	Branch main do feedback-infra	Só os serviços usados pelo projeto
`feedback-github-lambda-*`	Branch main de cada repositório de função	Só `lambda:UpdateFunctionCode` na própria função

Proteção dos dados (LGPD)

- Dados criptografados em repouso (KMS) e em trânsito (a API só aceita HTTPS); backup contínuo (PITR).
- O `userId` vem do token, não do corpo, e não entra no índice do relatório.
- Logs sem dados pessoais. E-mails dos administradores ficam em secrets do GitHub e são mascarados nos logs da pipeline.
- Texto do aluno escapado nos e-mails em HTML.

7. Monitoramento

A aplicação é monitorada pelo **Amazon CloudWatch**, com o que a AWS coleta automaticamente, sem custo extra.

O quê	Onde no CloudWatch	Para que serve

Logs das funções	`/aws/lambda/feedback-ingestion`
	`/aws/lambda/feedback-notification`
	`/aws/lambda/feedback-report`
		Ver cada execução e os erros (retenção de 14 dias, sem dados pessoais)

Métricas das funções	`AWS/Lambda`: Invocations, Errors, Duration, Throttles; IteratorAge no Streams	Quantas vezes rodou, quantas falharam, quanto tempo levou
Métricas da API	`AWS/ApiGateway`: Count, 4xx, 5xx, Latency	Volume, erros e tempo de resposta
Métricas do banco	`AWS/DynamoDB`: capacidade consumida, requisições limitadas	Uso e gargalos da tabela
Avisos que falharam	`AWS/SQS`: mensagens visíveis na `feedback-notification-dlq`	Mostra se algum aviso crítico não foi enviado

Como consultar: console da AWS → CloudWatch → *Log groups* ou *Metrics*; ou Lambda → função → aba *Monitor* (gráficos prontos). Pelo terminal: `aws logs tail /aws/lambda/feedback-ingestion --follow`.

Exemplos de linhas de log: `Saved evaluation <id>` · `Evaluation <id>: notified 2 admins` · `Weekly report 2026-11-02 to 2026-11-08: 5 evaluations, sent to 2 admins`.

8. Deploy

Deploy automatizado

Repositório	Evento	O que a pipeline faz
feedback-infra	Pull request	`terraform fmt -check`, `validate` e `plan`
feedback-infra	Push na main	`terraform apply`
3 repositórios de função	Pull request	Build, testes, ArchUnit, cobertura mínima de 80%
3 repositórios de função	Push na main	Testes e publicação com `aws lambda update-function-code`

Configuração no GitHub

Nome	Tipo	Onde	Conteúdo
`TF_API_TOKEN`	secret	feedback-infra	Token do Terraform Cloud
`SES_SENDER_EMAIL`	secret	feedback-infra	E-mail remetente
`ADMIN_EMAILS`	secret	feedback-infra	Lista JSON com os e-mails dos administradores
`AWS_PLAN_ROLE_ARN` / `AWS_APPLY_ROLE_ARN`	variable	feedback-infra	Saídas `infra_plan_role_arn` e `infra_apply_role_arn`
`AWS_ROLE_ARN`	variable	cada função	Item da saída `function_pipeline_role_arns`

Passo a passo do zero

1. Primeiro apply (local, uma vez) — as roles das pipelines ainda não existem, então usa credenciais de administrador da AWS:

```
# terraform.tfvars (fora do Git):
#   ses_sender_email = "remetente@exemplo.com"
#   admin_emails     = ["admin@exemplo.com"]
terraform login && terraform init && terraform apply
```

2. Clicar no link de verificação que o SES envia ao remetente e a cada administrador (conta no sandbox do SES). **3.** Configurar o GitHub (tabela acima). **4.** Push na main dos 3 repositórios de função para publicar o código.

5. Criar os usuários

```
POOL=$(terraform output -raw cognito_user_pool_id)
aws cognito-idp admin-create-user --user-pool-id "$POOL" --username aluno@exemplo.com \
  --user-attributes Name=email,Value=aluno@exemplo.com Name=email_verified,Value=true \
  --message-action SUPPRESS
aws cognito-idp admin-set-user-password --user-pool-id "$POOL" --username aluno@exemplo.com \
  --password '<senha>' --permanent
aws cognito-idp admin-add-user-to-group --user-pool-id "$POOL" --username aluno@exemplo.com \
  --group-name students        # admins: --group-name admins
```

6. Testar — nota de 0 a 3 dispara o e-mail de urgência:

```
CLIENT=$(terraform output -raw cognito_app_client_id)
API=$(terraform output -raw api_url)
TOKEN=$(aws cognito-idp admin-initiate-auth --user-pool-id "$POOL" --client-id "$CLIENT" \
  --auth-flow ADMIN_USER_PASSWORD_AUTH \
  --auth-parameters USERNAME=aluno@exemplo.com,PASSWORD='<senha>' \
  --query AuthenticationResult.IdToken --output text)

curl -X POST "${API%/}/evaluations" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"description": "O áudio da aula cortou", "score": 2}'

aws lambda invoke --function-name feedback-report out.json   # relatório sem esperar a segunda
```